Beyond Reported F1: A Shortcut-Controlled Evaluation of Program-Graph PHP Web-Shell Detection and the Limited Role of Learned Taint

Anh Hoang[1*] and Hoang Gia Dinh[1]

[1*]University of Economics Ho Chi Minh City (UEH), Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): anhh@ueh.edu.vn;
Contributing authors: giadh@ueh.edu.vn;

Abstract

Purpose: Learned PHP web-shell detectors routinely report near-ceiling scores, yet such scores do not show that a detector has learned malicious program behavior: source collections, benign coding style, file statistics, and payload packaging can all act as shortcuts. **Methods:** We contribute a shortcut-controlled evaluation protocol that combines near-duplicate-aware, cluster-disjoint partitions; input canonicalization; shallow and graph-metadata probes reported as baselines; a character-matched diagnostic subset; static deobfuscation; a two-sided obfuscation matrix that transforms benign and malicious files alike and separates structure-preserving from packaging transformations; a source-disjoint holdout; and precision at low deployment prevalences. We apply it to HiPaG, a hierarchical program-graph model (statement, function, file) with a learned taint-dependency predictor. **Results:** HiPaG reaches $\mathbf{98.48 \pm 0.27}$ macro-F1 on the in-distribution test split, but a random forest on shallow character statistics already reaches $\mathbf{96.81 \pm 0.21}$ and a fine-tuned CodeBERT reaches $\mathbf{99.00 \pm 0.51}$: the in-distribution score is largely shortcut-accessible and the best of it needs no program structure. Under the source shift HiPaG has the highest mean PR-AUC ($\mathbf{94.89 \pm 0.54}$), but after Holm correction its advantage over CodeBERT is not significant ($\boldsymbol{p}_{\text{Holm}} = \mathbf{0.053}$) and the hierarchy is indistinguishable from a flat graph baseline ($\boldsymbol{p}_{\text{Holm}} = \mathbf{0.065}$); on base-rate precision CodeBERT is ahead. A controlled ablation isolates the one component with a large effect: disabling static deobfuscation leaves clean and structure-preserving levels unchanged but collapses all three packaging levels (PR-AUC $\mathbf{97.49 \pm 1.49}$ to $\mathbf{28.87 \pm 2.57}$ at L2), so robustness to payload packaging comes from a preprocessing step, not from the architecture. Learned taint edges show no detectable difference from

rule edges or from no semantic edges, and equivalence tests do not confirm them equal either. **Conclusion:** Headline F1 is not evidence of program understanding. Web-shell detection results should be reported with shortcut probes, transformation families separated, an explicit statement of which preprocessing is applied, source shift and base-rate precision.

Keywords: PHP web-shell detection, Shortcut learning, Evaluation methodology, Program graphs, Taint analysis, Obfuscation

1 Introduction

Web shells are a documented, high-severity threat. The MITRE ATT&CK framework catalogues web-shell deployment as technique T1505.003, national agencies have issued dedicated guidance on detecting and preventing web-shell malware,[1] and web-shell deployment is a documented stage of actively exploited vulnerabilities such as CVE-2021-26855 (ProxyLogon).[2] The underlying weaknesses—unrestricted file upload (CWE-434), code injection (CWE-94) and OS command injection (CWE-78)—remain long-standing entries in CWE and the OWASP Web Security Testing Guide.

Learning-based PHP web-shell detectors have been proposed on lexical, opcode, syntax-tree and graph representations [1–3], and many report near-ceiling accuracy on their own datasets. A high score on a conventional split, however, does not reveal *what* the classifier has learned. Public web-shell collections and benign corpora differ in source, age, coding style, file size, character statistics, and in whether code is packed inside an `eval`/decoder wrapper. Each of these differences can be exploited by a classifier without any notion of malicious data flow—a manifestation of shortcut learning [12] and of the evaluation pitfalls documented for machine learning in security [13, 14].

This paper asks whether reported PHP web-shell detection performance remains informative once such shortcuts are controlled, and what a structurally motivated model actually gains under those controls. Our answer is deliberately measured. We find that *most of the in-distribution performance is attainable from shallow features*: a random forest on character statistics of canonicalized source comes within a few points of every learned model. On the in-distribution split a fine-tuned CodeBERT, which uses no program graph at all, matches or exceeds the hierarchical model on macro-F1 and in the low-false-positive regime alike, trailing it only on PR-AUC and by a margin well inside the seed-to-seed spread. Under source shift HiPaG ranks best on average, but once the pairwise comparisons are corrected for multiplicity no model is demonstrably more robust than another, and the hierarchy is indistinguishable from a flat graph baseline. The one component with an unmistakable effect is not part of the learned model at all: a controlled ablation shows that static deobfuscation alone decides whether packaged payloads are detected, while every architectural ablation moves accuracy by less than the seed variation. A learned taint-dependency predictor,

[1] NSA & ASD, *Detect and Prevent Web Shell Malware*, 2020.
[2] https://www.cisa.gov/news-events/cybersecurity-advisories/aa21-062a

which motivated the model, turns out to be indistinguishable from hand-written rules and from no semantic edges at the sample size we can afford.

Contributions.

- **A shortcut-controlled evaluation protocol** for PHP web-shell detection (Section 5, our main contribution). Each component targets a specific route to inflated scores: near-duplicate and source leakage, coding style, coarse file and parser statistics, character distributions, the "wrapper equals malware" shortcut, and balanced-split optimism. Shortcut probes are reported as baselines, and obfuscation is applied to both classes and split into structure-preserving and packaging families.
- **HiPaG**, a hierarchical multi-relational program-graph model (statement → function → file) with a token-sequence transformer, intra-function GATv2, an inter-procedural graph transformer, static deobfuscation, and a weakly supervised taint-dependency predictor with strict labels, permissive candidates and inter-procedural bridges (Section 4). We describe the mechanism without claiming that it determines performance.
- **An honest account of learned taint.** Against a pre-specified falsification criterion, learned semantic edges cannot be distinguished from hand-written rule edges or from no semantic edges at any level; the one level that passes the point gate has a confidence interval containing zero, and equivalence tests do not license the opposite claim either (Section 7). We report this null directly, with its power limits, together with the baselines we could not reproduce.
- **Packaging robustness is preprocessing, not architecture.** A controlled ablation that toggles only the static deobfuscator shows it is the single component with a large effect: without it all three packaging levels collapse to near-chance ranking, while every architectural ablation stays within seed variation (Section 7.5). Comparisons across models that differ in preprocessing cannot attribute packaging robustness to a representation, and we show the confound explicitly.

2 Related Work

2.1 PHP web-shell detection

Early learned detectors treat a script as a sequence of opcodes or tokens and classify it with convolutional or recurrent networks, a line summarized in a recent survey [2]; more recent work handles long scripts with sliding-window attention [7]. Structural detectors operate on abstract syntax trees, for example AST features with a deep forest [3], or on control-flow graphs. The nearest neighbour to our model is GlareShell [1], which builds a flat interprocedural control-flow graph, attaches hand-assigned risk levels to statements and classifies it with a graph neural network. Work on evasion studies obfuscated and generated web shells and multimodal feature fusion [4]; concurrent work co-evolves web-shell generation with multi-view detection for robustness [10] and benchmarks fine-grained web-shell family classification [9]. Published detectors are typically evaluated on their own datasets and splits, so their reported numbers are not directly comparable with each other or with ours; cross-dataset generalization in

particular is rarely evaluated, including for recent large-language-model detectors [8], and we therefore compare only against baselines run on our split (Section 6).

2.2 Program graphs and taint

Static taint analysis for PHP tracks user-controlled sources to sensitive sinks with hand-written rules [22]. Code property graphs merge syntax, control and data flow into one multi-relational graph [23], and graph neural networks over such program graphs have been used for program understanding [24] and vulnerability detection [25]. Our encoder uses GATv2 [26] and a graph-transformer layer [27], and our taint predictor follows the link-prediction view of learning edges from node states [28].

2.3 Shortcuts and evaluation pitfalls in security ML

Shortcut learning describes models that succeed through spurious decision rules that do not transfer [12]. In security, Arp et al. catalogue pitfalls including sampling bias, data snooping, spurious correlations and base-rate neglect [13]; TESSERACT shows how spatial and temporal bias inflate malware-classifier results [14]; and leakage is a widespread cause of irreproducible machine-learning results [15], and shortcut cues have been documented directly in a neighbouring security task, encrypted network-traffic classification [16]. Code duplication inflates the measured performance of models of code [17], a concern also recognized for web-shell datasets [5]; learned vulnerability detectors lose much of their reported accuracy under realistic evaluation [18, 19]. The base-rate fallacy makes precision at realistic prevalence the decisive operational quantity [20], and precision–recall analysis is more informative than ROC analysis under imbalance [21]. Recent PHP web-shell detectors pursue directions such as feature fusion with federated learning [6] and incremental learning [11], but to our knowledge these evaluation concerns have not been operationalized together for PHP web-shell detection; this is the gap our protocol addresses.

3 Threat Model

Setting.

The unit of analysis is one PHP source file. The detector parses it, never executes it, and outputs a web-shell score; the target use is static scanning of repositories, uploads and deployment pipelines.

Adversary.

The adversary controls the content of the malicious file and applies semantics-preserving transformations to evade detection: identifier renaming, dead-code insertion, comment and whitespace changes, string splitting and concatenation, variable functions, and payload packaging with `base64_decode`, `gzinflate` or nested decoders inside `eval`. Benign developers may also pack, minify or encode legitimate code, so a wrapper alone is not evidence of malice.

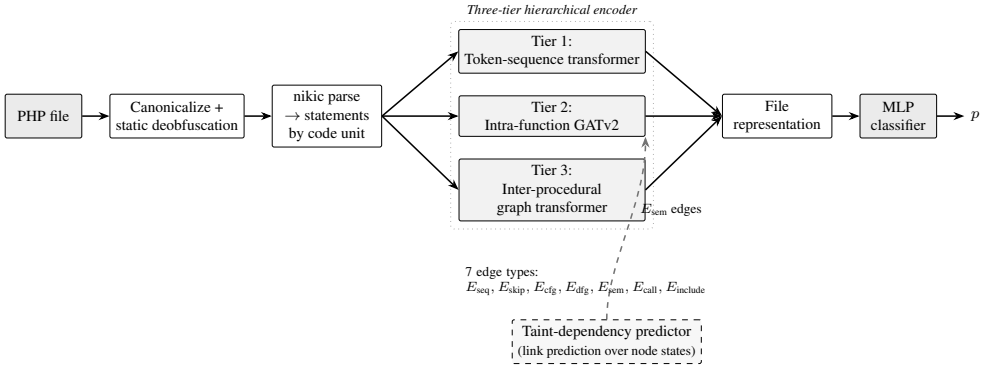


Fig. 1 HiPaG architecture. A canonicalized and statically deobfuscated PHP file is parsed into statements grouped by code unit; three tiers (token sequence, intra-function GATv2, inter-procedural graph transformer) produce a file representation, and the taint-dependency predictor supplies E_{sem} edges to tier two.

Out of scope.

We do not model a white-box adaptive adversary who optimizes inputs against the model, nor training-data poisoning, nor runtime behavior. Static analysis has a principled limit: when the payload exists only as data reconstructed at run time from inputs that are not available statically, no static detector can recover it. Payload *packaging* as we apply it is not that limit. A packaged payload is opaque only to a detector that does not unwrap it; the wrapper is constant-foldable, so a pipeline that decodes it statically recovers the original program, and one that does not sees a single opaque string. We therefore report packaging as a property of the *pipeline*, not as a boundary of static analysis. Every sample in our corpus is a single file; multi-file web shells linked by `include` are therefore not evaluated (Section 9).

Detection target.

The behavior of interest is a dependency from an attacker-controlled source (e.g. `$_POST`) to a code- or command-execution sink (e.g. `eval`, `system`). Whether a detector actually relies on this dependency, rather than on correlated surface cues, is exactly what the protocol of Section 5 is designed to test.

4 HiPaG

HiPaG lowers each file to a hierarchical multi-relational graph and classifies it with a three-tier encoder (Fig. 1). Semantic source-to-sink edges are produced in the forward pass by a taint-dependency predictor; the graph builder only stores candidate pairs and weak labels. We describe the mechanism in full, but the experiments in Section 7 show that the predictor is not what determines detection performance.

4.1 Parsing, canonicalization and static deobfuscation

Files are parsed with nikic/PHP-Parser.[3] Before parsing, source text is canonicalized at the token level: comments and documentation comments are removed, a trailing closing tag is dropped, and line endings are normalized inside whitespace tokens; string contents, heredocs and inline HTML are untouched. The model and the shallow probe see the same canonical text (Section 5.2). Files that fail to parse are kept with a minimal graph and an explicit parse status rather than silently removed.

A static deobfuscator unwraps common packaging without executing anything. It constant-folds expressions built from string literals, concatenation of literals, and nested calls to a fixed table of decoders (base64, gzip family, rot13, hex, URL and uu decoding), folding from the innermost call outwards. A narrow constant propagation substitutes variables that are assigned exactly once from a literal in the same scope. A decoded string is accepted as code only if it parses without error and yields at least one real statement; the resulting statements are grafted into the graph and flagged as deobfuscated. Decoding depth and intermediate sizes are bounded, and gzip-family inputs are decompressed incrementally so that a decompression bomb is never materialized. Arguments that are not compile-time constants, for example values read from a request, are left untouched; this is the static limit discussed in Section 3.

4.2 Hierarchical program graph

For a file we build statement nodes and code-unit nodes (one per function or method plus the top-level script), with seven relation types: E_{seq} (consecutive statements), E_{skip} (short forward skips within a small window), E_{cfg} (control flow from the structured statements of each code unit), E_{dfg} (def–use edges from reaching definitions), E_{sem} (predicted source-to-sink dependencies), E_{call} (calls to functions defined in the same file), and E_{include} (include relations between code units). The control-flow graph is approximate (no exception or `goto` edges), and array-element and property writes count as definitions of the base variable. Because each sample in our corpus is a standalone file, include targets outside the file cannot be resolved and E_{include} is empty in all reported experiments; the file level thus reduces to the call graph.

Statement features comprise a one-hot statement kind; multi-hot flags for seed sources, sinks and decoders; a flag for statements that contain both a source and a sink (e.g. `eval($_POST['c'])`); character entropy, length and relative position; and the statement's token sequence. A small vector of file-level indicators (entropy and the presence of `eval`-like calls, decoders and variable functions) is passed to the classification head.

4.3 Seeds: strict labels, permissive candidates

Sources are the PHP superglobals and request-reading APIs, including legacy input arrays. Sinks are split into two lists. *Strict* sinks (code and command execution, dynamic callables, `unserialize`, and context-dependent sinks: `include`/`require` with a non-constant argument and `preg_replace` with the `/e` modifier) are the only sinks

[3] https://github.com/nikic/PHP-Parser

used to create positive labels. *Candidate-only* sinks (file write, upload move, network and mail functions, `fopen` in a write mode, `extract`, and `ob_start` with a variable callback) may form candidate pairs but never labels. A sink name counts only when it is the callee of an actual call node or language construct in the syntax tree; string literals or identifiers that merely spell a sink name, as in `$f = 'assert'`, are not sinks. Calls through a variable are mapped to a dynamic-call sink.

4.4 Candidate pairs and weak labels

Within a code unit, a *generic* candidate is any ordered statement pair (i, j) with j reachable from i within $H = 12$ hops over $E_{\text{seq}} \cup E_{\text{cfg}} \cup E_{\text{dfg}}$. A *typed* candidate additionally has a source at i and a strict or candidate-only sink at j. Typed candidates also cross code units through two bridges bounded by $D = 2$ call hops: an *argument bridge* links a source in a caller to a sink in a callee when the tainted value is passed as the argument whose parameter reaches the sink, and a *return bridge* links a source in a callee whose tainted value reaches a `return` to a sink in the caller that consumes the call result. A bridge is *exact* when call arguments, parameters and def–use information are all available, and a *fallback* otherwise; no bridge is created merely because one function calls another. The candidate set is capped at $C_{\max} = 4096$ pairs per file, keeping typed candidates first and filling the remainder by uniform sampling of generic candidates.

Positive labels are only (source, strict sink) pairs connected by an in-unit path or an exact bridge. Negatives are sampled at $\rho = 5$ per positive, half from non-positive candidates (hard negatives) and half from other pairs; typed pairs are never used as negatives, because a typed pair without a label may still be a true dependency. These labels mark *reachable* seed pairs, not verified dynamic taint flows.

4.5 Taint-dependency predictor

For statement states h_i, h_j after tier one, the predictor scores a directed candidate,

$$p_{ij} = \sigma(\text{MLP}([\, h_i \| h_j \,])) \,. \tag{1}$$

An edge $i \to j$ enters E_{sem} when p_{ij} exceeds $\max(\tau, \tau_{\min})$ and is among the k highest-scoring candidates with destination j. The threshold τ is recalibrated on validation data after each epoch to maximize edge-level F1 against the weak labels, with floor $\tau_{\min} = 0.30$; the per-sink budget is $k = 8$. Selected edges carry p_{ij} as an edge attribute, so the detection loss also reaches the predictor through these weights. Training minimizes

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \lambda \, \mathcal{L}_{\text{taint}}, \qquad \lambda = 0.50, \tag{2}$$

where $\mathcal{L}_{\text{taint}}$ is a binary cross-entropy over positive and negative pairs whose positive weight is set per batch to $N_{\text{neg}}/N_{\text{pos}}$; without it, probabilities collapse toward the class prior, fall below $\tau_{\min}$ and E_{sem} becomes empty. For the first $W = 3$ epochs tier two receives the rule edges (positive pairs restricted to the candidate set and the same budget) while the predictor is trained; afterwards the predicted edges replace them. The same rule edges form the *rule* ablation, so the learned-versus-rule comparison

Table 1 Shortcuts that can inflate web-shell detection scores and the protocol component that controls or exposes each one.

Shortcut	Control or exposure
Near-duplicates across partitions; source identity	Canonical-token near-duplicate clustering; cluster-disjoint split stratified by label and source; source-disjoint holdout
Coding style, comments, line endings	Canonicalization applied identically to model and probes
Coarse file and parser statistics	Shallow and graph-metadata probes, reported as baselines
Size and entropy gaps between classes	Distribution-matched benign sampling; character-matched diagnostic subset
"Packed code is malicious"	Packed and minified benign variants; two-sided obfuscation matrix
Packaging hides structure	Static deobfuscation; packaging reported as a separate family
Balanced-split optimism	PR-AUC, FPR at fixed TPR, precision at low prevalence
Seed luck and post-hoc claims	Multiple seeds; pre-specified falsification criterion

is made over an identical candidate universe. It tests the value of learned *scoring* of candidates, not of candidate generation beyond static reachability.

4.6 Three-tier encoder

Tier one embeds each statement's hashed token sequence with a transformer encoder [29] and masked mean pooling, then concatenates the result with the statement features. *Tier two* applies GATv2 [26] over E_{seq}, E_{skip}, E_{cfg}, E_{dfg} and E_{sem} jointly; each edge carries a one-hot relation type and a weight (p_{ij} for semantic edges, one otherwise), so attention is relation-aware. *Tier three* max-pools statements into code units and applies graph-transformer layers [27] over E_{call} and E_{include} with root connections. Code units are max-pooled into a file vector, concatenated with the file-level indicators, and classified by a two-layer MLP.

5 Evaluation Protocol

The protocol is organized around one question: *which shortcuts could produce the reported score without the detector modelling malicious program behavior, and how is each one controlled or exposed?* Table 1 maps each shortcut to its control; the subsections explain why each control is needed.

5.1 Partitioning and source holdout

Exact duplicates are removed after canonicalization (groups with conflicting labels are discarded entirely), and near-duplicates are grouped by MinHash locality-sensitive hashing [33] over canonical token shingles, confirmed by estimated Jaccard similarity and merged into clusters. Train, validation and test partitions are assigned by cluster and stratified by label and dominant source, so variants of one shell never appear on both sides [17]. A separate collection is held out entirely; clusters of that collection that touch the training pool are removed. The holdout is source-disjoint but belongs to the same broad family of PHP web shells, so it measures transfer between collections, not cross-family generalization.

5.2 Canonicalization

Comments, documentation blocks, trailing tags and line-ending conventions differ systematically between benign projects and web-shell collections and can be learned as labels. Canonicalizing the text seen by every model and probe removes these cues by construction. A practical consequence is that comment insertion, a common "obfuscation level" in prior evaluations, becomes a no-op and is not reported as a robustness result.

5.3 Shortcut probes as baselines

Two random forests [30] quantify how much discrimination is available without program semantics. The *shallow* probe uses byte- and character-level statistics of the canonical text (size, line structure, entropy, character-class fractions, long base64-like runs, tag and namespace usage); it deliberately excludes sink and source names. The *graph-metadata* probe uses only by-products of parsing and graph construction (parse status and counts of statements, functions, edges, roles and candidates). Probes are evaluated with cluster-grouped cross-validation and with the same train-to-test, train-to-holdout and per-obfuscation-level transfers as the models, and they are reported as baselines alongside the models in the detection, holdout and per-level obfuscation tables (Tables 2 and 4). A strong probe is a warning, not a competitor: it shows how much of a score could be explained without the reasoning a structural model claims.

5.4 Distribution matching and the character-matched subset

Benign files are sampled with weights that match the joint distribution of file size and byte entropy of the training web shells, and a hard-benign subset contains legitimate code that uses sinks or decoders. A fraction of benign files are added as minified or packed variants so that packing is not a class marker. As a stricter diagnostic, the character-matched subset pairs structured web shells one-to-one with nearest-neighbour benign files on character features (entropy, digit fraction, log size, base64 presence) within a caliper, so that the shallow probe is close to chance on it. Matching never uses graph or taint information.

5.5 Two-sided obfuscation matrix

Test files of *both* classes are transformed at levels L0–L5 and never used for training or model selection. Structure-preserving levels change surface form while keeping control and data flow: L1 renames identifiers and inserts dead code, and L4 rewrites direct calls as variable-function calls built from string fragments. Packaging levels move the program into a string that is decoded at run time: L2 (base64), L3 (gzinflate) and L5 (multi-layer). L0 is the untransformed test set. Transforming benign files as well is essential: if only web shells were wrapped, a detector could score well on packaging levels simply by flagging wrappers. A transformation that breaks a file that originally passed the PHP linter is recorded and excluded. Structure-preserving levels test whether a detector uses structure; packaging levels test whether its pipeline unwraps constant-folded payloads (Section 7.5).

5.6 Metrics and statistical reporting

We report macro-F1, accuracy, precision, recall, MCC, PR-AUC, FPR at 95% TPR, and precision at deployment prevalences of 10%, 1% and 0.1%. Base-rate precision is computed from the operating point at the target TPR, so it is a prevalence adjustment rather than an observed precision on a deployment stream [20]. Every configuration is trained with several seeds (counts are given with each table) and reported as mean and sample standard deviation.

5.7 Pre-specified falsification criterion

Before training we fixed a criterion for the claim that learned semantic edges matter: the claim is rejected if the full model exceeds the no-semantic-edge model by less than 0.50 macro-F1 points at every obfuscation level with overlapping confidence intervals across seeds. Fixing the criterion in advance prevents the selective reporting of a favorable level.

6 Experimental Setup

Data.

Web shells come from three public collections (Tennc, JohnTroony, Cycle183); benign files come from a broad set of open-source PHP applications and libraries pinned to fixed commits. The partitions contain 1525/3787 (web shell/benign) training, 328/829 validation and 328/824 test files, with 160 hard-benign files in the test split. The c01dsnap collection (339 web shells, 1967 benign files) forms the source-disjoint holdout. Both of its classes come from the same upstream release: the web shells from its malicious archives and the benign files from a fixed random sample of its "White" PHP archive, drawn with a recorded seed and used only in the holdout, never in training. Keeping both holdout classes from one unseen source is deliberate — a holdout that reused our training benign pool would measure source shift on one class only — but it also means the benign side shifts together with the malicious side, so the holdout measures transfer to a different corpus rather than to a different threat. Its prevalence differs from the test split, so its macro-F1 is not directly comparable and base-rate precision is the more meaningful cross-split quantity.

Baselines run.

On the same split we run the shallow and graph-metadata probes, a bidirectional LSTM [31] over hashed parser tokens, and a flat GATv2 over the same statement graph without hierarchy or semantic edges. The latter two are controlled local baselines, not reproductions of named systems. We also fine-tune CodeBERT [32] (`microsoft/codebert-base`) on the same canonical text and the same split: files longer than the encoder limit are read as sliding windows of 512 tokens with stride 384, of which only the first eight are kept, so the tail of the longest 37.61% of training files is never seen. The file representation is the mean, across a file's windows, of the per-window pooled `[CLS]` vector (the encoder's `pooler_output`: a dense projection of `[CLS]` followed by tanh), not a mean over token states. Training uses AdamW at

a learning rate of 2×10^{-5}, with the checkpoint selected by validation PR-AUC and early stopping (patience 5, at most 25 epochs), over the same 5 seeds.

Baselines not run.

An opcode-CNN baseline was not run because a PHP opcode dumper was not available in our environment; GlareShell [1] was not reproduced because its implementation, its manually assigned risk dictionary and its graph preprocessing are not available, and our rule-edge ablation is not a GlareShell reproduction; YARA was not run because no pinned PHP web-shell rule set and executable were available. These omissions limit the comparison and are not evidence that HiPaG outperforms those systems.

Training.

Models are trained with AdamW; the checkpoint is selected by validation PR-AUC with early stopping, never in the warm-up phase. Taint hyper-parameters (k and W, with $\tau_{\min}$ fixed) were chosen on validation macro-F1 by a separate sweep before any test evaluation. Complete hyperparameters for every model are listed in Appendix B; the full run manifest (library versions, data hashes and per-seed fingerprints) is released as `repro/manifest.json` with the code.

Research questions.

RQ1: how does HiPaG compare with the probes and local baselines on the test split? **RQ2**: how do the full, rule-edge and no-semantic-edge variants behave across obfuscation levels (RQ4 checkpoints, inference only)? **RQ3**: how well do models transfer to the source-disjoint holdout? **RQ4**: which components matter in a clean-split ablation? **RQ5**: which taint hyper-parameters does the validation sweep select?

7 Results

7.1 Shortcut probes

The probes confirm that the task, as commonly posed, is largely shortcut-solvable. Under cluster-grouped cross-validation the shallow probe reaches a ROC-AUC of $99.49 \pm 0.02\%$, and trained on the training split it reaches $99.51 \pm 0.03\%$ on test and $97.04 \pm 0.12\%$ on the holdout; the graph-metadata probe reaches $99.38 \pm 0.03\%$ on test. The probes also behave as the transformation families predict. Variable-function rewriting barely moves the character statistics, and the shallow probe keeps an ROC-AUC of $99.43 \pm 0.03\%$ at L4 ($96.78 \pm 0.13\%$ at L1). Under packaging, where both classes are wrapped, the shallow probe's AUC falls sharply to $65.17 \pm 0.23\%$ (L2) and $64.88 \pm 0.21\%$ (L5). The graph-metadata probe does *not* collapse in the same way ($91.18 \pm 0.22\%$ at L2 and $84.70 \pm 0.40\%$ at L5), because the static deobfuscation of Section 4.1 restores enough structure for the parser-derived counts to stay discriminative; this is a property of the pipeline, not evidence that the detector reasons about flow. The static deobfuscation is directly responsible: with it disabled, every packaged file reduces to the same one-statement wrapper graph, so graph metadata carries no information and the probe is at chance under packaging ($50.00 \pm 0.00\%$ at L2,

Table 2 RQ1 and RQ3: shared test split and source-disjoint holdout (mean ± sample SD over 5 seeds; all metrics except MCC in %). Shallow and graph-metadata RFs are shortcut probes. FPR@95: FPR at 95% TPR; P@π: precision at prevalence π.

Split	Model	Macro-F1	MCC	PR-AUC	FPR@95	P@1%	P@0.1%
Test	Shallow RF	96.81 ± 0.21	0.936 ± 0.004	98.72 ± 0.06	1.60 ± 0.22	37.74 ± 3.05	5.69 ± 0.68
	Graph-meta RF	96.65 ± 0.16	0.933 ± 0.003	98.45 ± 0.05	1.82 ± 0.09	34.59 ± 1.10	4.98 ± 0.23
	Bi-LSTM	96.73 ± 0.72	0.935 ± 0.014	98.51 ± 0.67	2.65 ± 1.54	31.51 ± 14.39	4.80 ± 3.02
	Flat GATv2	97.96 ± 0.93	0.959 ± 0.018	99.28 ± 0.39	0.83 ± 0.34	55.49 ± 11.12	11.84 ± 5.35
	CodeBERT	99.00 ± 0.51	0.980 ± 0.010	99.70 ± 0.25	0.17 ± 0.20	87.03 ± 14.25	57.73 ± 39.81
	HiPaG	98.48 ± 0.27	0.970 ± 0.005	99.82 ± 0.07	0.27 ± 0.22	80.54 ± 14.39	41.24 ± 34.75
Holdout	Shallow RF	89.06 ± 0.27	0.781 ± 0.005	89.82 ± 0.43	14.68 ± 0.55	6.16 ± 0.21	0.65 ± 0.02
	Graph-meta RF	88.36 ± 0.30	0.776 ± 0.006	84.87 ± 0.14	55.37 ± 1.05	1.73 ± 0.04	0.17 ± 0.00
	Bi-LSTM	88.24 ± 0.78	0.767 ± 0.016	85.53 ± 2.53	39.74 ± 5.28	2.40 ± 0.29	0.24 ± 0.03
	Flat GATv2	91.41 ± 0.32	0.833 ± 0.007	93.00 ± 1.68	13.82 ± 5.54	7.44 ± 3.11	0.80 ± 0.36
	CodeBERT	90.93 ± 2.32	0.821 ± 0.046	83.25 ± 6.71	25.45 ± 41.69	10.41 ± 5.73	1.17 ± 0.66
	HiPaG	92.06 ± 1.19	0.850 ± 0.019	94.89 ± 0.54	14.65 ± 6.65	7.25 ± 3.25	0.78 ± 0.37

$50.00 \pm 0.00\%$ at L5), whereas on clean data the two branches are almost identical ($99.15 \pm 0.06\%$ without versus $99.38 \pm 0.03\%$ with deobfuscation). A learned detector that merely matches the shallow probe on L0 and L4 therefore provides no evidence of structural reasoning by those numbers alone.

7.2 RQ1: shared-split comparison

Table 2 reports the main metrics; the full metric set is in Appendix A. HiPaG reaches 98.48 ± 0.27 macro-F1 on the test split, but it is not the best model there: a fine-tuned CodeBERT reaches 99.00 ± 0.51, while the shallow probe already reaches 96.81 ± 0.21 and the flat GATv2 97.96 ± 0.93. *The in-distribution score does not require program structure:* the spread across these very different hypothesis classes is a few points of macro-F1, and a pretrained code language model with no program graph at all sits at the top of it. The low-false-positive regime separates the models more sharply, but it does not single out HiPaG either. HiPaG's FPR at 95% TPR (0.27 ± 0.22) is lower than that of the flat GATv2 (0.83 ± 0.34) and of the shallow probe (1.60 ± 0.22), and its estimated precision at 1% prevalence (80.54 ± 14.39) is correspondingly higher; CodeBERT, however, matches or exceeds HiPaG on both (0.17 ± 0.20 and 87.03 ± 14.25), with seed spreads wide enough that we do not read a winner from either column. HiPaG retains a slight edge in PR-AUC (99.82 ± 0.07 against 99.70 ± 0.25). At 0.1% prevalence all estimates are low and highly variable across seeds, for HiPaG and CodeBERT alike, so we draw no ranking from them.

7.3 RQ3: source-disjoint holdout

All models lose performance on the c01dsnap holdout (Table 2, lower block). HiPaG has the highest mean macro-F1 (92.06 ± 1.19) and the highest mean PR-AUC (94.89 ± 0.54, against 93.00 ± 1.68 for the flat GATv2 and 89.82 ± 0.43 for the shallow probe), but the pairwise comparisons do not establish it as the better model. We compare HiPaG against each baseline on holdout PR-AUC with two-sided Welch tests and Holm correction within the family; the correction is post hoc, so we report both raw and adjusted values. Against CodeBERT the gap is large in the mean but does not survive correction ($p_{\text{raw}} = 0.018$, $p_{\text{Holm}} = 0.053$), and against the flat GATv2 — the variant

without hierarchy — it is not significant even before correction ($p_{\text{raw}} = 0.065$, $p_{\text{Holm}} = 0.065$). On this holdout the hierarchy buys nothing we can measure, and HiPaG's advantage over the strongest in-distribution model is not statistically supported.

CodeBERT does have the lowest mean and lowest median holdout PR-AUC among the six models (mean 83.25 ± 6.71, median 81.98%), but this ranking is fragile and we do not treat it as established. Its seeds range from 76.07% to 94.35%, overlapping the next-lowest model, and one seed degenerates entirely at the operating point (FPR at 95% TPR of 1.0); we retain all five seeds and report the spread rather than removing it. What the holdout does show robustly is instability: CodeBERT's FPR at 95% TPR (25.45 ± 41.69) varies across seeds by more than its own mean, so it supports no stable operating point.

HiPaG's own low-false-positive advantage largely disappears as well: its FPR at 95% TPR (14.65 ± 6.65) is comparable to those of the shallow probe and the flat GATv2. On the base-rate quantities we argued are the more meaningful cross-split measure (Section 5.6), HiPaG is in fact *behind* CodeBERT: precision at 1% prevalence is 7.25 ± 3.25 against 10.41 ± 5.73, and at 0.1% prevalence 0.78 ± 0.37 against 1.17 ± 0.66, with neither difference significant. At the default threshold HiPaG's loss is mainly in recall (77.17 ± 4.55) rather than precision (97.92 ± 2.60). The structural model therefore ranks somewhat better on average off-distribution, but after correction no model is demonstrably more robust than another, and nothing here would justify a claim of reliable deployment detection on unseen sources.

7.4 RQ2: obfuscation matrix

Table 3 reports macro-F1 and PR-AUC per level for the three semantic-edge variants (full metrics in Appendix A), and Fig. 2 plots macro-F1 by level. Two patterns stand out. First, behavior depends on the transformation family rather than on a single difficulty scale: the structure-preserving levels L1 and L4 cost a few points, whereas the packaging levels L2 and L5 cost much more and show very large variation across seeds, even though both classes are wrapped and static deobfuscation is applied. Table 4 extends this matrix to every baseline, and Section 7.5 shows that the deobfuscation step itself, not the architecture, is what keeps these levels from collapsing altogether. The gzinflate level L3 is the exception among packaging levels and remains comparatively high. Second, learned semantic edges do not provide a consistent benefit. The no-semantic-edge variant has an equal or higher mean macro-F1 than the full model at L2, L3 and L4. The full model has the higher mean at L1, where it also leads the rule variant, and its leads at L0 and L5 are small relative to the seed variation. Learned and rule edges alternate in rank across levels, again within seed variation.

Falsification criterion.

The pre-specified criterion required the full model to exceed the no-semantic-edge model by at least 0.50 pp of macro-F1 at *every* level. Only L1 meets the point estimate (0.99 pp), and its unadjusted Welch CI95 is $[-1.57, 3.56]$ pp, which contains zero; at every other level the point estimate is below the margin or negative. The criterion is therefore not met, and the single level that passes the point gate is a deviation we record rather than a result we rely on.

Table 3 RQ2: macro-F1 and PR-AUC (%) per obfuscation level for the full model (learned E_{sem}), rule E_{sem} and no E_{sem} (mean ± sample SD over 5 seeds; inference on RQ4 checkpoints). S: structure-preserving; P: packaging. Both classes are transformed.

		Macro-F1			PR-AUC		
Level	Kind	Full	Rule	None	Full	Rule	None
L0	–	98.48 ± 0.27	98.54 ± 0.38	98.28 ± 0.66	99.82 ± 0.07	99.76 ± 0.10	99.74 ± 0.25
L1	S	95.38 ± 2.00	93.74 ± 2.50	94.39 ± 1.37	98.67 ± 0.86	97.61 ± 0.87	97.54 ± 1.29
L4	S	87.63 ± 3.14	87.77 ± 1.94	88.52 ± 2.57	90.07 ± 2.88	88.28 ± 3.64	89.84 ± 2.23
L2	P	78.67 ± 16.83	76.92 ± 13.47	80.49 ± 13.08	97.49 ± 1.49	96.72 ± 1.32	96.68 ± 1.72
L3	P	94.49 ± 3.52	97.09 ± 2.23	96.96 ± 1.48	99.14 ± 0.34	99.26 ± 0.45	99.31 ± 0.43
L5	P	66.42 ± 20.25	61.53 ± 13.94	65.97 ± 14.54	86.83 ± 4.35	82.36 ± 5.23	84.44 ± 3.72

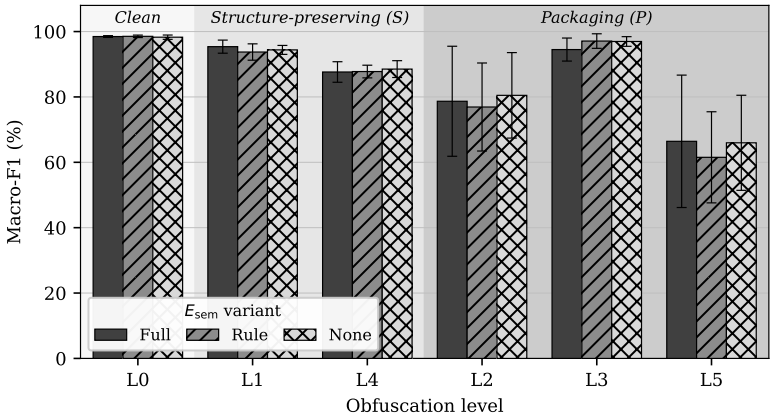


Fig. 2 Macro-F1 by obfuscation level for full, rule-edge and no-semantic-edge variants (mean and seed SD), grouped by structure-preserving and packaging levels. Generated from the same results as Table 3.

Two further checks bound what this null can mean. With five seeds per group the minimum difference detectable at 80% power is 1.10 pp of macro-F1 on the clean split, rising to 36.24 pp at the noisiest packaging level, so the design is underpowered against effects smaller than the margin itself. Conversely, two one-sided tests against a 0.50 pp equivalence margin reject equivalence at no level. We therefore report the accurate conclusion: *at this sample size there is no detectable difference between learned semantic edges, rule edges and no semantic edges, and the data do not establish that the three are equivalent either.* We do not claim that learned taint edges help, and we do not claim they are useless; we claim the experiment cannot tell them apart.

7.5 Static deobfuscation is the component that matters

Table 4 shows a clean split at the packaging levels: the shallow probe and CodeBERT collapse (PR-AUC 24.83 ± 3.01 and 51.29 ± 0.43 at L2), while the parser- and graph-derived models stay high (96.31 ± 0.95 for the Bi-LSTM and 94.45 ± 5.60 for the flat GATv2). This split is *confounded*: the models that survive packaging are exactly the models whose pipeline applies the static deobfuscator, and the models that collapse

Table 4 RQ2 extended: every model under the two-sided obfuscation matrix (mean ± sample SD over 5 seeds, in %). "Deobf." states whether that model's input pipeline applies the static deobfuscator: the graph- and parser-derived models consume deobfuscated program graphs, while the shallow probe and CodeBERT read canonicalized source text only. L1 and L4 are structure-preserving; L2, L3 and L5 package the payload. Baseline rows are inference runs on the saved RQ1 checkpoints; the HiPaG row is the RQ2 run on the same obfuscated split and seeds.

Metric	Model	Deobf.	L1	L2	L3	L4	L5
PR-AUC	Shallow RF	No	93.30 ± 0.31	51.29 ± 0.43	64.21 ± 0.16	98.57 ± 0.06	51.53 ± 0.54
	Graph-meta RF	Yes	97.03 ± 0.10	74.50 ± 0.88	74.50 ± 0.88	70.13 ± 0.38	57.41 ± 0.74
	Bi-LSTM	Yes	88.60 ± 5.08	96.31 ± 0.95	97.06 ± 0.93	89.08 ± 3.80	83.77 ± 7.04
	Flat GATv2	Yes	96.30 ± 0.88	94.45 ± 5.60	99.06 ± 0.18	87.50 ± 8.12	82.30 ± 8.10
	CodeBERT	No	96.68 ± 2.49	24.83 ± 3.01	45.44 ± 2.02	99.56 ± 0.43	24.54 ± 2.98
	HiPaG (full)	Yes	98.67 ± 0.86	97.49 ± 1.49	99.14 ± 0.34	90.07 ± 2.88	86.83 ± 4.35
Macro-F1	Shallow RF	No	77.09 ± 0.49	34.78 ± 2.65	71.60 ± 0.18	96.58 ± 0.12	34.18 ± 2.87
	Graph-meta RF	Yes	95.26 ± 0.17	73.76 ± 0.38	73.76 ± 0.38	84.33 ± 0.25	66.29 ± 0.48
	Bi-LSTM	Yes	87.24 ± 3.58	89.14 ± 1.32	93.79 ± 0.94	85.76 ± 2.77	78.38 ± 3.83
	Flat GATv2	Yes	93.60 ± 0.67	85.36 ± 9.67	96.62 ± 0.30	80.37 ± 2.52	67.14 ± 8.27
	CodeBERT	No	96.55 ± 0.97	30.27 ± 10.14	49.21 ± 9.97	98.79 ± 0.47	29.94 ± 9.99
	HiPaG (full)	Yes	95.38 ± 2.00	78.67 ± 16.83	94.49 ± 3.52	87.63 ± 3.14	66.42 ± 20.25

are exactly those reading canonicalized text. The comparison therefore cannot tell a representation effect from a preprocessing effect, and we do not read it as evidence that program graphs beat text under packaging.

A controlled ablation separates the two. We retrain HiPaG with static deobfuscation disabled and change nothing else, over 5 seeds (Table 5). On clean input the two configurations are indistinguishable (99.82 ± 0.07 with deobfuscation against 99.70 ± 0.08 without), and they stay close at both structure-preserving levels, L1 and L4. At the three packaging levels the model without deobfuscation collapses to near or below chance ranking (28.87 ± 2.57 at L2, 33.11 ± 6.14 at L3 and 29.02 ± 2.36 at L5) while the same architecture with deobfuscation retains 97.49 ± 1.49, 99.14 ± 0.34 and 86.83 ± 4.35. This is by a wide margin the largest effect of any component we ablated: no architectural ablation in Section 7.6 moves macro-F1 by more than the seed variation, whereas removing the deobfuscator costs roughly seventy points of PR-AUC at L2 and L5. *Robustness to payload packaging in this system is produced by the static deobfuscation step, not by the hierarchy, the graph representation or the learned taint edges.*

Two caveats keep this from being read as a pipeline-quality measurement. Every transformed file was produced successfully at every level (100.00% transform success), and every packaged file parsed (100.00% parse success at L2, L3 and L5, against 98.44% at L0, L1 and L4, where a small tail falls back to token features), so the collapse without deobfuscation is not a parsing artifact. However, our artifacts do not record a per-level count of how often the deobfuscator actually recovered the wrapped payload: the deobfuscation flag we log belongs to the base graph build, not to the obfuscated levels. We therefore report the effect of enabling the component, not its per-level success rate, and the deobfuscator's own accuracy on these wrappers remains unquantified.

Table 5 Deobfuscation ablation: the same HiPaG architecture with the static deobfuscator enabled and disabled (PR-AUC and macro-F1 in %, mean ± sample SD over 5 seeds). Only the packaging levels L2, L3 and L5 are affected.

Metric	Level	With deobf.	Without deobf.
PR-AUC	L0	99.82 ± 0.07	99.70 ± 0.08
	L1	98.67 ± 0.86	96.27 ± 2.21
	L2	97.49 ± 1.49	28.87 ± 2.57
	L3	99.14 ± 0.34	33.11 ± 6.14
	L4	90.07 ± 2.88	90.62 ± 5.98
	L5	86.83 ± 4.35	29.02 ± 2.36
Macro-F1	L0	98.48 ± 0.27	98.18 ± 0.60
	L1	95.38 ± 2.00	89.90 ± 4.01
	L2	78.67 ± 16.83	22.16 ± 0.00
	L3	94.49 ± 3.52	41.89 ± 0.28
	L4	87.63 ± 3.14	84.08 ± 3.16
	L5	66.42 ± 20.25	22.16 ± 0.00

Table 6 RQ4: clean-split ablation on the test split (mean ± sample SD over 5 seeds; all metrics except MCC in %).

Variant	Macro-F1	MCC	PR-AUC	FPR@95	P@1%
Full (learned E_{sem})	98.48 ± 0.27	0.970 ± 0.005	99.82 ± 0.07	0.27 ± 0.22	80.54 ± 14.39
Rule E_{sem}	98.54 ± 0.38	0.971 ± 0.008	99.76 ± 0.10	0.29 ± 0.18	78.47 ± 12.93
No E_{sem}	98.28 ± 0.66	0.966 ± 0.013	99.74 ± 0.25	0.17 ± 0.18	86.82 ± 13.72
No E_{dfg}	98.57 ± 0.43	0.971 ± 0.008	99.85 ± 0.07	0.19 ± 0.18	85.14 ± 13.86
No token transformer	98.21 ± 0.40	0.964 ± 0.008	99.63 ± 0.19	0.32 ± 0.24	77.13 ± 11.88
No inter-procedural tier	98.61 ± 0.43	0.972 ± 0.009	99.78 ± 0.06	0.24 ± 0.09	80.42 ± 5.69
No file-level features	98.41 ± 0.33	0.968 ± 0.006	99.75 ± 0.12	0.17 ± 0.07	85.34 ± 4.88
Mean pooling	98.11 ± 0.54	0.962 ± 0.011	99.32 ± 0.33	0.51 ± 0.13	65.86 ± 6.22

7.6 RQ4: component ablation

Table 6 shows the clean-split ablation; the full metric set is in Appendix A (Table A3). No single removal lowers mean macro-F1 by more than the seed variation; removing the inter-procedural tier, data-flow edges or replacing learned with rule edges leaves the mean equal or higher. Learned semantic edges make no measurable difference on clean data: the full model (98.48 ± 0.27), the rule variant (98.54 ± 0.38) and the variant without semantic edges (98.28 ± 0.66) are indistinguishable within seed variation. The one component whose removal consistently degrades the ranking-based and base-rate metrics is max pooling: mean pooling gives the lowest PR-AUC and precision at low prevalence. This suggests that the decisive signal is often localized in a few statements, which a max-pooled hierarchy preserves.

7.7 RQ5: taint hyper-parameters

The validation sweep over the per-sink budget k and the warm-up length W ($\tau_{\min}$ fixed) selected $k = 8$ and $W = 3$, with validation macro-F1 98.47 ± 0.40 over 3 seeds. The spread across the grid was within seed variation, consistent with the small

influence of the semantic edges reported above. This is a selection result, not a test-set estimate.

7.8 Character-matched diagnostic

On the character-matched subset (54 pairs) the shallow probe reaches a ROC-AUC of only $57.70 \pm 5.24\%$, so character cues are largely removed; the graph-metadata probe still reaches $67.72 \pm 2.12\%$, so the subset does *not* meet its preset gate of low discrimination for both probes, because parser-derived structural counts still separate the classes. Only 15 of its files fall into the test split. On them the full, rule and no-semantic-edge variants reach macro-F1 of 97.00 ± 6.71, 98.56 ± 3.21 and 100.00 ± 0.00, respectively, but with six positives and nine negatives the seed standard deviations are several points wide. The caliper (0.56) was selected post hoc as the largest value keeping the shallow-probe AUC at or below the 60% gate, before any model was evaluated on the subset; no caliper in the sweep satisfies the full gate (which also requires a low graph-metadata AUC and a minimum file count). This subset therefore neither supports nor refutes any comparative claim, and we report it only as a diagnostic.

8 Discussion

Most of the score is shortcut-accessible.

Under a cluster-disjoint split with canonicalized input, a random forest on character statistics comes within a few macro-F1 points of every learned model. Reporting a near-ceiling macro-F1 on such data therefore says little about whether a detector models malicious behavior. The protocol makes the remaining, more informative differences visible: low-FPR behavior, source transfer, and the split between structure-preserving and packaging transformations.

What the structural model adds.

In-distribution, HiPaG's advantage over the *shortcut probes* is concentrated where false positives matter: a lower FPR at high recall and higher precision at low prevalence than the shallow probe and the flat GATv2. That advantage is not specific to program structure, however: a fine-tuned CodeBERT, which uses no graph at all, matches or exceeds HiPaG in the same low-false-positive regime (Section 7.2). On the source-disjoint holdout HiPaG's edge over the probes in the low-false-positive regime disappears, and what remains is a higher mean ranking score that does not survive multiplicity correction against CodeBERT and is not significant against the flat graph baseline even before it. On the base-rate quantities we called the more meaningful cross-split measure, CodeBERT is ahead. We therefore do not claim that the structural model transfers better; we claim the holdout separates models less than the test split appears to. On structure-preserving transformations that leave character statistics intact (L4), the shallow probe keeps its discrimination, so these levels alone cannot isolate structural reasoning; the character-matched subset was designed for that purpose but is too small at test time to be conclusive.

Why learned taint makes no measurable difference here.

Several mechanisms plausibly explain the null result; we state them as hypotheses rather than findings. (i) The information that the semantic edges encode is already available: statement features flag seed sources, sinks and decoders, a dedicated feature marks same-statement source-to-sink cases, and the file-level indicators record `eval`-like calls and decoders. (ii) Candidates are generated by the same static reachability that defines the weak labels, so the predictor mostly re-scores pairs that the rule edges already connect; it is not given the opportunity to recover dependencies that static analysis misses. (iii) Identifier renaming (L1) is the one setting in which re-scoring could help, because learned statement states are less tied to identifier names, and it is the only level showing a (statistically unconfirmed) advantage. Decoupling candidate generation from static reachability is a natural next step, but we do not claim it would change the outcome.

Packaging tests the pipeline, not the model.

Whether a detector survives payload packaging is decided by one preprocessing step. The controlled ablation in Section 7.5 holds the architecture fixed and toggles only the static deobfuscator: clean and structure-preserving levels are unaffected, while all three packaging levels collapse to near-chance ranking without it. No architectural component we ablated comes close to that effect. Read across models, the same pattern explains Table 4: the systems that keep their scores under packaging are the ones whose pipeline unwraps the payload, not the ones with richer representations. Packaging is therefore not a boundary of static analysis and not a property of the model class; it is a question of whether the pipeline constant-folds the wrapper. Evaluations that wrap only malicious files would reward wrapper detection, and evaluations that average packaging with structure-preserving levels would hide the effect entirely. We recommend reporting the two families separately and stating explicitly whether deobfuscation is applied.

9 Threats to Validity

Construct validity.

Source, sink and decoder roles come from finite seed lists; reachability-based weak labels are not verified taint flows, and unlisted APIs or indirect behavior are missed. A statement that contains both source and sink yields no inter-statement positive pair; such cases reach the model only through a node feature. The semantic-edge ablation therefore evaluates this particular learned-edge design, not taint analysis in general. The control-flow and data-flow graphs are approximate.

Internal validity.

The local baselines are controlled re-implementations, not reproductions of published systems, and GlareShell, opcode-CNN and YARA are absent for the reasons in Section 6. Taint hyper-parameters were selected on validation data, and RQ2 reuses RQ4 checkpoints without retraining. The character-matched subset was constructed by searching over the matching caliper until the shallow probe approached chance, so

its operating point was chosen after inspecting probe results; together with its small test portion this makes it a diagnostic only. An exploratory analysis on files containing both a source and a sink is not reported because that slice overlapped the training partition. The CodeBERT baseline is also handicapped by its own input budget: the eight-window cap discards the tail of the longest 37.61% of training files, so a configuration that read whole files might transfer differently on the holdout. We did not run that configuration and claim no direction for it.

External validity.

The corpora are public collections of historical PHP web shells and open-source benign code; private or recent web shells may differ. The holdout is source-disjoint but from the same broad family, and every sample is a single file, so neither cross-family transfer nor multi-file web shells joined through `include` are tested. The transformations are fixed rather than adaptive, and our packaging levels test whether the pipeline unwraps a constant-folded payload rather than marking a boundary of static analysis; a payload reconstructed at run time from request data would lie beyond any static detector, and we do not evaluate that case.

Conclusion validity.

Seed aggregates are reported as mean and sample standard deviation over five seeds. Pairwise comparisons use two-sided Welch tests with Holm correction inside each family, and the semantic-edge contrasts additionally report two one-sided equivalence tests and the minimum detectable effect at 80% power. Both the correction and the equivalence margin were fixed after the runs, so they are post hoc; we report raw and adjusted p-values side by side rather than only the favourable one. With five seeds per group the design is underpowered for differences near our 0.50 pp margin, so "no measurable difference" means exactly that and never "proven equivalent". Base-rate precision is derived from a single operating point and inherits its uncertainty; at the lowest prevalence it is highly variable across seeds.

10 Conclusion

We proposed a shortcut-controlled evaluation protocol for PHP web-shell detection and used HiPaG, a hierarchical program-graph model with a learned taint-dependency predictor, as the instrument to exercise it. Four findings follow. In distribution, the headline score is largely shortcut-accessible: a character-statistics random forest comes within a few points of every learned model, and a fine-tuned CodeBERT with no program graph scores highest. Under source shift HiPaG ranks best on average, but after multiplicity correction no model is demonstrably more robust than another and the hierarchy is indistinguishable from a flat graph baseline. Robustness to payload packaging is produced by static deobfuscation rather than by the architecture: toggling that one preprocessing step moves the packaging levels between near-chance and near-ceiling, while no architectural ablation leaves the seed noise. And the learned taint edges that motivated the model cannot be distinguished from hand-written rules or from their own removal, with the power to detect only differences larger than the

margin we set out to test. Taken together, a headline F1 certifies neither program understanding nor robustness. We recommend that web-shell detection studies report shortcut probes, two-sided and family-separated obfuscation results, an explicit statement of which preprocessing each compared system applies, source-disjoint evaluation with corrected pairwise tests, and base-rate precision alongside headline scores.

Data and code availability. The analyzed collections are identified by their upstream sources and pinned commits; redistribution is subject to the original licenses. Code, configuration files, split manifests and the evaluation-protocol scripts will be released with the publication, excluding non-redistributable data.

Reproducibility. Every reported number is a mean ± sample standard deviation over 5 seeds (0–4), except the taint hyper-parameter sweep, which used 3 seeds. The environment was Python 3.12.16, PyTorch 2.15.0 (CUDA 13.2 build), PyTorch Geometric 2.8.1 and Transformers 4.57.6, with PHP 8.5.4 and nikic/PHP-Parser 5.9.0 for parsing; training ran on one NVIDIA RTX 2080 Super Max-Q (8 GB) with driver 580.178.04 and cuDNN 9.26.0, on an Intel Core i7-10875H. Runs are seeded through `random`, `numpy`, `torch` and `torch.cuda`; HiPaG runs additionally set `cudnn.deterministic` and disable `cudnn.benchmark`, while the CodeBERT baseline runs under AMP `fp16`. We did not enable `torch.use_deterministic_algorithms`, and `torch-scatter` is not installed, so some PyTorch Geometric scatter reductions on the GPU are order-dependent. We therefore do not claim bit-exact reruns, and report dispersion across seeds instead of single-run figures. The CodeBERT baseline is `microsoft/codebert-base` at revision `3b0952fe`, with 512-token windows at stride 384 capped at eight per file, averaged pooled `[CLS]` vectors, AdamW at 2×10^{-5}, and the checkpoint chosen by validation PR-AUC with early stopping (patience 5, at most 25 epochs). HiPaG uses AdamW with the same checkpoint rule; its taint hyper-parameters ($k = 8$, $W = 3$, with $\tau_{\min} = 0.30$ fixed) were chosen by a validation macro-F1 sweep. A full manifest — library versions, data hashes, split manifests and per-seed run fingerprints — is released with the code.

Ethical considerations and dual use. All web-shell samples were treated as inert data. Files were parsed and canonicalized statically with a PHP parser and never executed; no PHP runtime interpreted any payload at any stage, and all processing ran in an isolated environment rather than on any live server. The corpora are existing public collections used under their original licenses; we neither collected new live web shells nor deployed any sample. We did not test third-party systems, so no responsible-disclosure process applies. Web-shell detection is dual-use: the same study describes an obfuscation matrix and a static deobfuscator. We release the detector, the evaluation protocol and the deobfuscation routines, which are defensive. The obfuscation levels are built from standard, publicly documented techniques (base64 and gzip packing, variable-function rewriting, identifier renaming and dead-code insertion); we release them as evaluation fixtures bound to our corpora, for reproducing the protocol, rather than as a general-purpose web-shell generator, and we add no new malicious payloads beyond transformations of samples already public.

Declarations

- **Funding.** This study was funded by the University of Economics Ho Chi Minh City (UEH).
- **Competing interests.** The authors have no competing interests to declare that are relevant to the content of this article.
- **Data availability.** The datasets are drawn from public collections and are subject to their original licenses; processed data are not redistributed here.

Appendix A Full metric tables

Table A1 RQ1 and RQ3, all metrics (mean ± sample SD over 5 seeds; all metrics except MCC in %). Shallow and graph-metadata RFs are shortcut probes.

Split	Model	Macro-F1	Acc.	Prec.	Rec.	MCC	PR-AUC	FPR@95	P@10%	P@1%	P@0.1%
Test	Shallow RF	96.81 ± 0.21	97.41 ± 0.17	96.28 ± 0.02	94.57 ± 0.59	0.936 ± 0.004	98.72 ± 0.06	1.60 ± 0.22	86.88 ± 1.55	37.74 ± 3.05	5.69 ± 0.68
Test	Graph-meta RF	96.65 ± 0.16	97.27 ± 0.13	95.30 ± 0.33	95.12 ± 0.22	0.933 ± 0.003	98.45 ± 0.05	1.82 ± 0.09	85.32 ± 0.60	34.59 ± 1.10	4.98 ± 0.23
Test	Bi-LSTM	96.73 ± 0.72	97.36 ± 0.57	96.75 ± 1.02	93.90 ± 2.43	0.935 ± 0.014	98.51 ± 0.67	2.65 ± 1.54	80.90 ± 9.49	31.51 ± 14.39	4.80 ± 3.02
Test	Flat GATv2	97.96 ± 0.93	98.35 ± 0.74	97.95 ± 0.74	96.22 ± 2.35	0.959 ± 0.018	99.28 ± 0.39	0.83 ± 0.34	92.83 ± 2.75	55.49 ± 11.12	11.84 ± 5.35
Test	CodeBERT	99.00 ± 0.51	99.18 ± 0.42	98.56 ± 1.40	98.60 ± 0.67	0.980 ± 0.010	99.70 ± 0.25	0.17 ± 0.20	98.45 ± 1.83	87.03 ± 14.25	57.73 ± 39.81
Test	HiPaG	98.48 ± 0.27	98.77 ± 0.22	98.65 ± 0.93	97.01 ± 1.25	0.970 ± 0.005	99.82 ± 0.07	0.27 ± 0.22	97.61 ± 1.93	80.54 ± 14.39	41.24 ± 34.75
Holdout	Shallow RF	89.06 ± 0.27	94.49 ± 0.13	80.99 ± 0.45	81.71 ± 0.47	0.781 ± 0.005	89.82 ± 0.43	14.68 ± 0.55	41.91 ± 0.89	6.16 ± 0.21	0.65 ± 0.02
Holdout	Graph-meta RF	88.36 ± 0.30	94.73 ± 0.12	91.59 ± 0.56	70.62 ± 0.68	0.776 ± 0.006	84.87 ± 0.14	55.37 ± 1.05	16.19 ± 0.33	1.73 ± 0.04	0.17 ± 0.00
Holdout	Bi-LSTM	88.24 ± 0.78	94.38 ± 0.37	84.82 ± 1.96	75.28 ± 2.02	0.767 ± 0.016	85.53 ± 2.53	39.74 ± 5.28	21.22 ± 2.11	2.40 ± 0.29	0.24 ± 0.03
Holdout	Flat GATv2	91.41 ± 0.32	95.98 ± 0.17	93.53 ± 3.01	78.23 ± 2.34	0.833 ± 0.007	93.00 ± 1.68	13.82 ± 5.54	45.36 ± 10.94	7.44 ± 3.11	0.80 ± 0.36
Holdout	CodeBERT	90.93 ± 2.32	95.26 ± 1.35	82.12 ± 8.40	87.79 ± 1.19	0.821 ± 0.046	83.25 ± 6.71	25.45 ± 41.69	50.98 ± 23.37	10.41 ± 5.73	1.17 ± 0.66
Holdout	HiPaG	92.06 ± 1.19	96.38 ± 0.45	97.92 ± 2.60	77.17 ± 4.55	0.850 ± 0.019	94.89 ± 0.54	14.65 ± 6.65	44.44 ± 11.91	7.25 ± 3.25	0.78 ± 0.37

Table A2 RQ2, all metrics per obfuscation level (mean ± sample SD over 5 seeds; all metrics except MCC in %). L1 and L4 preserve structure; L2, L3 and L5 package the payload.

Variant	Level	Macro-F1	Acc.	Prec.	Rec.	MCC	PR-AUC	FPR@95	P@10%	P@1%	P@0.1%
Full	L0	98.48 ± 0.27	98.77 ± 0.22	98.65 ± 0.93	97.01 ± 1.25	0.970 ± 0.005	99.82 ± 0.07	0.27 ± 0.22	97.61 ± 1.93	80.54 ± 14.39	41.24 ± 34.75
Full	L1	95.38 ± 2.00	96.15 ± 1.76	91.31 ± 6.23	96.10 ± 2.04	0.910 ± 0.038	98.67 ± 0.86	2.84 ± 2.81	81.57 ± 16.09	40.40 ± 26.73	8.75 ± 8.32
Full	L2	78.67 ± 16.83	79.48 ± 17.23	66.86 ± 25.57	96.46 ± 6.76	0.672 ± 0.223	97.49 ± 1.49	2.69 ± 1.92	80.93 ± 10.37	31.37 ± 13.04	4.69 ± 2.72
Full	L3	94.49 ± 3.52	95.76 ± 2.50	97.97 ± 2.29	87.13 ± 10.63	0.896 ± 0.061	99.14 ± 0.34	1.65 ± 1.35	87.30 ± 8.63	43.02 ± 15.55	7.71 ± 4.17
Full	L4	87.63 ± 3.14	88.94 ± 3.16	73.35 ± 6.81	97.87 ± 2.34	0.776 ± 0.046	90.07 ± 2.88	8.86 ± 1.14	54.63 ± 3.12	9.92 ± 1.13	1.08 ± 0.14
Full	L5	66.42 ± 20.25	67.33 ± 20.58	52.68 ± 20.79	97.50 ± 4.64	0.519 ± 0.229	86.83 ± 4.35	14.93 ± 6.87	43.67 ± 10.04	6.87 ± 2.43	0.73 ± 0.27
Rule	L0	98.54 ± 0.38	98.82 ± 0.30	98.82 ± 0.51	97.01 ± 0.82	0.971 ± 0.008	99.76 ± 0.10	0.29 ± 0.18	97.37 ± 1.65	78.47 ± 12.93	37.39 ± 35.26
Rule	L1	93.74 ± 2.50	94.72 ± 2.25	87.87 ± 7.05	95.30 ± 2.16	0.878 ± 0.046	97.61 ± 0.87	4.54 ± 2.02	70.98 ± 9.63	19.74 ± 7.93	2.46 ± 1.22
Rule	L2	76.92 ± 13.47	77.71 ± 14.04	59.78 ± 16.28	99.33 ± 0.40	0.637 ± 0.177	96.72 ± 1.32	6.80 ± 5.02	65.07 ± 17.90	18.36 ± 12.77	2.41 ± 2.06
Rule	L3	97.09 ± 2.23	97.69 ± 1.67	97.31 ± 1.12	94.57 ± 7.03	0.943 ± 0.041	99.26 ± 0.45	0.90 ± 0.35	92.33 ± 2.67	53.34 ± 9.25	10.64 ± 3.65
Rule	L4	87.77 ± 1.94	89.10 ± 1.92	73.06 ± 3.62	98.23 ± 1.21	0.778 ± 0.030	88.28 ± 3.64	9.66 ± 0.70	52.46 ± 1.77	9.13 ± 0.59	0.99 ± 0.07
Rule	L5	61.53 ± 13.94	62.14 ± 14.11	44.47 ± 9.30	99.33 ± 1.00	0.450 ± 0.144	82.36 ± 5.23	22.33 ± 9.68	34.57 ± 10.35	4.82 ± 2.15	0.50 ± 0.23
None	L0	98.28 ± 0.66	98.61 ± 0.52	99.00 ± 0.55	96.10 ± 1.77	0.966 ± 0.013	99.74 ± 0.25	0.17 ± 0.18	98.45 ± 1.67	86.82 ± 13.72	57.14 ± 40.25
None	L1	94.39 ± 1.37	95.43 ± 1.10	92.20 ± 3.31	91.89 ± 3.84	0.889 ± 0.027	97.54 ± 1.29	5.07 ± 2.42	68.83 ± 10.22	18.10 ± 7.09	2.21 ± 1.04
None	L2	80.49 ± 13.08	81.37 ± 13.55	64.78 ± 17.36	98.60 ± 1.25	0.684 ± 0.178	96.68 ± 1.72	6.41 ± 5.05	66.35 ± 17.71	19.91 ± 15.75	2.81 ± 2.97
None	L3	96.96 ± 1.48	97.57 ± 1.15	97.60 ± 0.57	93.78 ± 4.35	0.940 ± 0.028	99.31 ± 0.43	0.95 ± 0.20	91.87 ± 1.62	51.04 ± 5.36	9.49 ± 1.83
None	L4	88.52 ± 2.57	89.83 ± 2.52	75.04 ± 6.20	97.56 ± 2.23	0.790 ± 0.040	89.84 ± 2.23	8.71 ± 1.42	55.18 ± 3.89	10.14 ± 1.37	1.11 ± 0.16
None	L5	65.97 ± 14.54	66.68 ± 14.99	48.42 ± 11.70	98.96 ± 1.82	0.501 ± 0.155	84.44 ± 3.72	18.45 ± 7.35	38.52 ± 9.81	5.63 ± 2.24	0.59 ± 0.25

Table A3 RQ4, all metrics for the clean-split ablation on the test split (mean ± sample SD over 5 seeds; all metrics except MCC in %).

Variant	Macro-F1	Acc.	Prec.	Rec.	MCC	PR-AUC	FPR@95	P@10%	P@1%	P@0.1%
Full (learned E_{sem})	98.48 ± 0.27	98.77 ± 0.22	98.65 ± 0.93	97.01 ± 1.25	0.970 ± 0.005	99.82 ± 0.07	0.27 ± 0.22	97.61 ± 1.93	80.54 ± 14.39	41.24 ± 34.75
Rule E_{sem}	98.54 ± 0.38	98.82 ± 0.30	98.82 ± 0.51	97.01 ± 0.82	0.971 ± 0.008	99.76 ± 0.10	0.29 ± 0.18	97.37 ± 1.65	78.47 ± 12.93	37.39 ± 35.26
No E_{sem}	98.28 ± 0.66	98.61 ± 0.52	99.00 ± 0.55	96.10 ± 1.77	0.966 ± 0.013	99.74 ± 0.25	0.17 ± 0.18	98.45 ± 1.67	86.82 ± 13.72	57.14 ± 40.25
No E_{dfg}	98.57 ± 0.43	98.84 ± 0.34	98.42 ± 0.99	97.50 ± 2.00	0.971 ± 0.008	99.85 ± 0.07	0.19 ± 0.18	98.25 ± 1.66	85.14 ± 13.86	54.06 ± 42.03
No token transformer	98.21 ± 0.40	98.54 ± 0.33	97.80 ± 0.85	97.07 ± 1.02	0.964 ± 0.008	99.63 ± 0.19	0.32 ± 0.24	97.15 ± 2.05	77.13 ± 11.88	28.12 ± 11.49
No inter-procedural tier	98.61 ± 0.43	98.87 ± 0.35	98.71 ± 0.54	97.32 ± 1.30	0.972 ± 0.009	99.78 ± 0.06	0.24 ± 0.09	97.80 ± 0.77	80.42 ± 5.69	30.16 ± 8.42
No file-level features	98.41 ± 0.33	98.72 ± 0.26	99.02 ± 0.84	96.46 ± 1.73	0.968 ± 0.006	99.75 ± 0.12	0.17 ± 0.07	98.44 ± 0.60	85.34 ± 4.88	37.90 ± 8.70
Mean pooling	98.11 ± 0.54	98.47 ± 0.43	98.14 ± 0.42	96.46 ± 1.47	0.962 ± 0.011	99.32 ± 0.33	0.51 ± 0.13	95.42 ± 1.16	65.86 ± 6.22	16.49 ± 3.99

Appendix B Hyperparameters and model selection

Table B4 lists the hyperparameters of every model. HiPaG and its taint predictor are configured from `configs/model/hipag_base.yaml`; the local baselines from `configs/experiment/rq1_baselines.yaml`; the CodeBERT baseline from `configs/experiment/rq1_codebert.yaml`. The taint budget k and warm-up length W are the values selected by the RQ5 validation sweep. All values are reproduced in the machine-readable `repro/manifest.json` released with the code.

Table B4 Hyperparameters per model. Neural models share AdamW with learning rate 2×10^{-4}, weight decay 10^{-4}, batch size 32, up to 100 epochs, and early stopping with patience 15 on validation PR-AUC; the CodeBERT baseline uses its own optimizer settings as noted.

Model	Settings
HiPaG (encoder)	Tier 1 token-sequence transformer: embedding 64, 4 heads, 2 layers. Tier 2 intra-function GATv2: hidden 128, 2 heads, 2 layers. Tier 3 inter-procedural graph transformer (UniMP): hidden 128, 2 heads, 2 layers. Dropout 0.1; global max pooling; file-level features {Shannon entropy, `eval` present, decoder present, variable-function present}.
HiPaG (taint predictor)	Link-prediction MLP with hidden size 128; per-sink budget $k = 8$; warm-up $W = 3$ epochs; probability floor $\tau_{\min} = 0.30$; edge loss weight $\lambda = 0.50$; `pos_weight` set automatically to the per-batch negative:positive ratio (5:1 by construction). Candidate pairs: within-12-hop reachability, inter-procedural depth 2, capped at 4096 per graph.
Flat GATv2	Statement-level GATv2 over sequence/skip/CFG/DFG edges, no hierarchy or semantic edges: hidden 64, 2 heads, 2 layers; token embedding 32; dropout 0.2; concatenated mean and max pooling.
Bi-LSTM	Bidirectional LSTM over hashed parser tokens: token embedding 64, hidden 96 per direction, vocabulary 4096, up to 512 tokens, dropout 0.25.
Shallow RF, Graph-metadata RF	Random forest with 300 trees, balanced class weights, fixed per-seed random state. Shallow features are character statistics of the canonical text; graph-metadata features are parser-derived structural counts.
CodeBERT	`microsoft/codebert-base` (revision `3b0952fe`); 512-token windows, stride 384, at most 8 windows per file, mean of per-window pooled `[CLS]` vectors. AdamW at 2×10^{-5}, weight decay 10^{-4}; checkpoint by validation PR-AUC with early stopping (patience 5, at most 25 epochs); AMP `fp16`.

References

[1] Feng, P., Wei, D., Li, Q., Wang, Q., Hu, Y., Xi, N., Ma, J.: GlareShell: Graph learning-based PHP webshell detection for web server of industrial internet. Computer Networks **245**, 110406 (2024). https://doi.org/10.1016/j.comnet.2024.110406

[2] Ma, M., Han, L., Zhou, C.: Research and application of artificial intelligence based webshell detection model: a literature review. arXiv:2405.00066 preprint (2024)

[3] Dong, C., Li, D.: AST-DF: a new webshell detection method based on abstract syntax tree and deep forest. Electronics **13**(8), 1482 (2024). https://doi.org/10.3390/electronics13081482

[4] Zhang, Y., Kang, H., Wang, Q.: MMFDetect: webshell evasion detection method based on multimodal feature fusion. Electronics **14**(3), 416 (2025). https://doi.org/10.3390/electronics14030416

[5] Hannousse, A., Nait-Hamoud, M.C., Yahiouche, S.: A deep learner model for multi-language webshell detection. International Journal of Information Security **22**(1), 47–61 (2023). https://doi.org/10.1007/s10207-022-00615-5

[6] Zeng, Q., Chai, J., Wu, J.: A webshell detection method based on feature fusion and federated learning. International Journal of Information Security **24**(4), 161 (2025). https://doi.org/10.1007/s10207-025-01078-0

[7] Wang, Z., Wang, H., Hao, L.: Poster: Long PHP webshell files detection based on sliding window attention. arXiv:2502.19257 preprint (2025)

[8] Han, F., Zhang, J., Deng, C., Tang, J., Liu, Y.: Can LLMs handle webshell detection? Overcoming detection challenges with a behavioral function-aware framework. arXiv:2504.13811 preprint (2025)

[9] Han, F.: Beyond detection: a comprehensive benchmark and study on representation learning for fine-grained webshell family classification. arXiv:2512.05288 preprint (2025)

[10] Ding, Y.: ShellForge: adversarial co-evolution of webshell generation and multi-view detection for robust webshell defense. arXiv:2601.22182 preprint (2026)

[11] Wang, Z., Wang, H., Yuan, S., Tian, Z.: Incremental learning research for webshell detection. Journal of King Saud University – Computer and Information Sciences **37**(8), 249 (2025). https://doi.org/10.1007/s44443-025-00235-8

[12] Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., Wichmann, F.A.: Shortcut learning in deep neural networks. Nature Machine Intelligence **2**(11), 665–673 (2020)

[13] Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: Proceedings of the 31st USENIX Security Symposium (2022)

[14] Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L.: TESSERACT: Eliminating experimental bias in malware classification across space and time. In: Proceedings of the 28th USENIX Security Symposium (2019)

[15] Kapoor, S., Narayanan, A.: Leakage and the reproducibility crisis in machine-learning-based science. Patterns **4**(9), 100804 (2023)

[16] Wang, C., Xie, X., Wang, T., Cui, Y.: Bias in the shadows: explore shortcuts in encrypted network traffic classification. arXiv:2601.10180 preprint (2026)

[17] Allamanis, M.: The adverse effects of code duplication in machine learning models of code. In: Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!) (2019)

[18] Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: Are we there yet? IEEE Transactions on Software Engineering **48**(9), 3280–3296 (2022)

[19] Risse, N., Böhme, M.: Uncovering the limits of machine learning for automatic vulnerability detection. In: Proceedings of the 33rd USENIX Security Symposium (2024)

[20] Axelsson, S.: The base-rate fallacy and the difficulty of intrusion detection. ACM Transactions on Information and System Security **3**(3), 186–205 (2000)

[21] Saito, T., Rehmsmeier, M.: The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE **10**(3), e0118432 (2015)

[22] Jovanovic, N., Kruegel, C., Kirda, E.: Pixy: A static analysis tool for detecting web application vulnerabilities. In: IEEE Symposium on Security and Privacy (2006)

[23] Yamaguchi, F., Golde, N., Arp, D., Rieck, K.: Modeling and discovering vulnerabilities with code property graphs. In: IEEE Symposium on Security and Privacy (2014)

[24] Allamanis, M., Brockschmidt, M., Khademi, M.: Learning to represent programs with graphs. In: International Conference on Learning Representations (ICLR) (2018)

[25] Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Advances in Neural Information Processing Systems (NeurIPS) (2019)

[26] Brody, S., Alon, U., Yahav, E.: How attentive are graph attention networks? In: International Conference on Learning Representations (ICLR) (2022)

[27] Shi, Y., Huang, Z., Feng, S., Zhong, H., Wang, W., Sun, Y.: Masked label prediction: unified message passing model for semi-supervised classification. In: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI) (2021)

[28] Zhang, M., Chen, Y.: Link prediction based on graph neural networks. In: Advances in Neural Information Processing Systems (NeurIPS) (2018)

[29] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS) (2017)

[30] Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)

[31] Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997)

[32] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M.: CodeBERT: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP (2020)

[33] Broder, A.Z.: On the resemblance and containment of documents. In: Proceedings of Compression and Complexity of Sequences (SEQUENCES) (1997)